

1=> Testing Database with 2 tables and 9 rows each. (Postgresql)

```
[postgres=# \c testing
You are now connected to database "testing" as user "adityadave".
[testing=# \dt
      List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | customers | table | adityadave
 public | orders    | table | adityadave
(2 rows)
```

```
[testing=# select * from customers;
 customer_id | name
-----+-----
           1 | Alice
           2 | Bob
           3 | Charlie
           4 | David
           5 | Emma
           6 | Frank
           7 | Grace
           8 | Henry
           9 | Dawe
(9 rows)
```

```
[testing=# select * from orders;
 order_id | customer_id
-----+-----
       101 |           1
       102 |           2
       103 |           3
       104 |           4
       105 |           5
       106 |           6
       107 |           7
       108 |           8
       109 |           1
(9 rows)
```

=> Enable Policy:

```
testing=# ALTER TABLE customers ENABLE ROW LEVEL SECURITY;
[ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE
ALTER TABLE
```

=> created normal UDF (without SECURITY DEFINER) for RLS policy:

```
testing=# CREATE OR REPLACE FUNCTION customer_predicate(cid INT)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
BEGIN
RETURN EXISTS (
SELECT 1
FROM orders o
WHERE o.customer_id = cid
);
END;
[$$;
CREATE FUNCTION
testing=# CREATE OR REPLACE FUNCTION orders_predicate(cid INT)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
BEGIN
RETURN EXISTS (
SELECT 1
FROM customers c
WHERE c.customer_id = cid
);
END;
[$$;
CREATE FUNCTION
testing=#
```

=> created RLS Policy

```
testing=# CREATE POLICY customer_policy
ON customers
FOR SELECT
USING (
customer_predicate(customer_id)
);
CREATE POLICY
testing=# CREATE POLICY orders_policy
ON orders
FOR SELECT
USING (
orders_predicate(customer_id)
);
CREATE POLICY
testing=#
```

=> Give permission to ROLE=alice.

```
testing=# GRANT SELECT ON customers TO alice;
[GRANT SELECT ON orders TO alice;
GRANT
GRANT
testing=# █
```

=> Cyclic dependency detected:

```
testing=> SELECT * FROM customers;
ERROR: stack depth limit exceeded
HINT: Increase the configuration parameter "max_stack_depth" (currently 2048kB), after ensuring the platform's stack depth limit is adequate.
CONTEXT: SQL expression "EXISTS (
SELECT 1
FROM orders o
WHERE o.customer_id = cid
)"
PL/pgSQL function customer_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
SELECT 1
FROM customers c
WHERE c.customer_id = cid
)"
PL/pgSQL function orders_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
SELECT 1
FROM orders o
WHERE o.customer_id = cid
)"
PL/pgSQL function customer_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
SELECT 1
FROM customers c
WHERE c.customer_id = cid
)"
PL/pgSQL function orders_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
SELECT 1
FROM orders o
WHERE o.customer_id = cid
)"
PL/pgSQL function customer_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
SELECT 1
FROM customers c
WHERE c.customer_id = cid
)"
PL/pgSQL function orders_predicate(integer) line 3 at RETURN
SQL expression "EXISTS (
```

Now the question is, can SECURITY DEFINER bypass RLS policy?

=> Now including SECURITY DEFINER in the function:

```
testing=# CREATE OR REPLACE FUNCTION customer_predicate(cid INT)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
RETURN EXISTS (
SELECT 1
FROM orders o
WHERE o.customer_id = cid
);
END;
$$;
CREATE FUNCTION
testing=# CREATE OR REPLACE FUNCTION orders_predicate(cid INT)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
RETURN EXISTS (
SELECT 1
FROM customers c
WHERE c.customer_id = cid
);
END;
$$;
CREATE FUNCTION
```

=> SECURITY DEFINER bypassed RLS policy!!!!!!


```
[testing=# SET ROLE alice;
SET
```

```
[testing=> SELECT * FROM customers;
```

customer_id	name
1	Alice
2	Bob
3	Charlie
4	David
5	Emma
6	Frank
7	Grace
8	Henry

```
(8 rows)
```

```
[testing=> SELECT * FROM orders;
```

order_id	customer_id
101	1
102	2
103	3
104	4
105	5
106	6
107	7
108	8
109	1

```
(9 rows)
```

```
testing=> █
```

Now we need to understand the reason behind it??????

Before that, I am pasting the result of sql server result.

Results	Messages	Query Plan	Plan Tree	Top Operations
customer_id		name		
1	1	Alice		
2	2	Bob		
3	3	Charlie		
4	4	David		
5	5	Emma		
6	6	Frank		
7	7	Grace		
8	8	Henry		
XML Showplan				
1	<ShowPlanXML xmlns="http://schemas.microsoft.com/sql...			

So, both results are the same. Now the main task is to understand how SECURITY DEFINER is internally making this circular policy execution possible.